Java Stream API – Practical Case Studies

Case Study 1 – Employee Grouping and Age Analysis (Case_1)

Create a List<Employee> with the fields: employee name, age, and department.

1. Group employees by department and display each department with its employees.
2. Group employees by age and display each age with the employees in that age group.
3. Find the eldest/oldest employee.
4. Find the youngest employee.

Case Study 2 – Two Lists, Numbers, and Strings (Case_2)

Create two lists of integers, list1 and list2.

5. Find the sum, average, maximum, and minimum of list1 and list2 separately.
6. Find the K most frequent elements in list1 and list2 separately. Ask for K as input or define it in the program.
7. Find the second most frequent element in each list separately. Decide how ties should be handled.
8. Merge the two lists into a single list using Stream API and flatMap().
9. Find the common elements between the two lists using streams.
10. Create a list of strings and create a Map in which each string is associated with its length.
11. Find all prime numbers from a given list using streams.
12. Find all non-prime numbers from a given list using streams. Decide whether 1 should be included as non-prime.
13. Given a string, find all repeating characters using streams.
14. Given a string, find the first repeating character using streams.
15. Find the longest word in a given list of strings.
16. Find the shortest word in a given list of strings.

Case Study 3 – Department-Wise Employee Salary Analysis

Create a list of Employee objects with these attributes: id, name, job, qualification, dateOfJoining, salary, and department.

17. Find the highest-paid employee in each department.
18. Find the second-highest-paid employee in each department.
19. Find the second-highest distinct salary in each department and the employee(s) earning it.
20. Display the department name along with the selected employee details.

Case Study 4 – Company and Employee Data (Case_4)

Create a List<Company>. Each Company contains a list of Employee objects.

Company fields: id, name, location, type (for example, FINANCE or SOFTWARE).

Employee fields: id, name, job, and salary.

21. Flatten the employee lists from all companies and collect a list of all employee names.
22. Find all employees whose salary is greater than ₹80,000 across all companies.
23. Find all unique job roles across all companies.
24. Find the total number of employees across all companies.
25. Find employees whose names start with 'A' or 'S' across all companies.

Case Study 5 – Patient Data Analysis (Case_5)

Create a collection of Patient objects with these attributes: patientId, name, department, doctorName, diagnosis, treatmentGiven, patientType (OUT, IN, ICU), heartRate, bloodPressure, sugarValue, and condition/severity.

26. Find all ICU patients.
27. Find all active ICU patients whose heart rate is greater than 100. Add a status field such as ACTIVE or DISCHARGED to the Patient class if active status is required.
28. Sort patients by seriousness, from most serious to least serious, using a clearly defined condition/severity score.

Case Study 6 – General Stream Operations (Case_6)

29. Create a list of strings. Filter strings with at least four characters, convert them to uppercase, remove duplicates, sort naturally, and collect the result. Display the output at each stage.
30. Given an integer array, keep values greater than 5, square them, remove duplicates, and sort in ascending order.
31. Given a list of integers, find all numbers that appear more than once and display their frequencies.
32. Given a string, find the first non-repeating character. Handle the case where no such character exists.

Case Study 7 – Advanced Employee Stream Operations

Create a list of Employee objects with at least: id, name, department, salary, and skills (List<String>).

33. Find the names of employees in the Engineering department whose salary is greater than ₹80,000, and sort their names alphabetically.
34. Extract all unique employee skills that start with the letter 'J'. Flatten the skills lists before removing duplicates.
35. Group employees by department, but include only employees whose salary is greater than ₹80,000. Omit departments with no matching employees.
36. Keep only the first employee encountered from each department. Preserve the original encounter order if that is part of the requirement.